

Fed-RWM: A Robust Watermarking Approach for Federated Learning Model Ownership Protection

Xin Tong¹, Student Member, IEEE, Shike Li, Graduate Student Member, IEEE, Ni Jin, and Baojiang Cui¹

Abstract—The interconnected nature of the Internet of Things (IoT) significantly enhances the efficiency of industries such as smart manufacturing, but it also raises concerns about data privacy. Federated learning (FL) utilizes an edge-cloud collaborative mode that shares models in the cloud instead of data, effectively mitigating data privacy leakage in edge IoT devices. However, FL suffers from the risk of model leakage, and both the cloud and the edge may illegally copy and sell the model, infringing on model ownership. To prevent such misbehavior, it is essential to design a robust method for verifying the model ownership. In this paper, we propose Fed-RWM, a novel watermarking method for FL models that provides robust ownership verification and avoids both edge and cloud leakage of watermark information. Fed-RWM trains the watermark by sharing parameters with an additional model and incorporates a watermark recovery method during verification to counter complex watermark removal attacks, which enhances the verification robustness. Fed-RWM introduces a new training paradigm that performs continuous watermarking training at the FL task initiator, preventing access to watermark information at both the edge and the cloud. The experimental results demonstrate that Fed-RWM performs well in model ownership verification and fidelity, is robust to different watermark removal attacks, and can provide reliable protection for federated learning models.

Index Terms—Internet of Things, federated learning, model watermarking, ownership verification.

I. INTRODUCTION

THE INTERNET of Things (IoT) enhances the interconnection of devices like smartphones, smart appliances, and vehicles by integrating technologies such as artificial intelligence (AI), 5G communications, and edge computing, thereby improving daily convenience. These devices collect vast amounts of private data daily, which are then transmitted to cloud servers for centralized processing. In practice, cloud servers are often insecure, posing a risk of raw data leakage [1], [2], [3]. Federated learning (FL) [4], a distributed machine learning framework, has been proposed as an alternative. In the FL system, multiple smart devices train the model locally, and only model parameters are exchanged with the cloud server, avoiding direct leakage of

raw data. This advantage makes FL widely applicable in IoT, including intrusion detection [5], smart healthcare [6], and smart manufacturing [7].

FL protects the original data privacy but introduces the risk of model leakage [8]. These models are collaboratively trained by task initiators and edge devices (e.g., factories, hospitals, banks) within the Internet of Things, consuming significant computational and data resources. The ownership of the resulting FL models resides with the FL group. FL model leakage refers to the unauthorized copying, distribution, or sale of FL models by untrustworthy entities (e.g., compromised servers or clients) to third parties not involved in the training process. Such third parties may deploy the stolen models on embedded devices (e.g., robots, smart vehicles, mobile phones) for commercial sale or offer online access for profit. This behavior seriously infringes on the ownership rights of the FL group, which can lead to substantial financial losses and undermine the fairness of the market for neural network models [9]. In the business environment, enterprises, as FL model producers, should act as the main body to maintain the legal use of the models, respect originality, reject plagiarism, and create a healthy market environment. Therefore, protecting the intellectual ownership of FL models is essential.

Model watermarking can effectively protect FL model ownership. It embeds a watermark into the FL model without affecting performance and creates an ownership verification mechanism to identify model attribution [10]. Existing FL model watermarking methods are categorized based on watermark embedding and training location into client-side and server-side watermarking. These methods assume that either the client-side or server-side is dishonest, with watermark embedding training completed on the honest side. The honest party explicitly knows the watermark embedding method and details (e.g., embedding position, watermark content). In reality, both the server and clients may be dishonest, making it easy to use embedding information to precisely remove the watermark and illegally sell the model. Moreover, while the dishonest party cannot access the embedding information, it can still launch several watermark removal attacks that disrupt the verification process, leading to failure [8], see Fig. 1. We found that current FL watermarking methods are based on [11] and not resilient to novel watermark elimination attacks, such as parameter scaling and hybrid attacks.

To address these issues, we propose a robust watermarking method, Fed-RWM, for protecting FL model ownership. We assign the watermark embedding and training tasks to the honest FL task initiator, thus preventing dishonest server-side

Received 25 December 2024; revised 10 June 2025; accepted 30 July 2025.
Date of publication 7 August 2025; date of current version 5 December 2025.
The associate editor coordinating the review of this article and approving it for publication was Y. Miao. (Corresponding author: Baojiang Cui.)

Xin Tong, Ni Jin, and Baojiang Cui are with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: cuibj@bupt.edu.cn).

Shike Li is with the School of Computer and Information Technology, Shanxi University, Taiyuan 237016, China.

Digital Object Identifier 10.1109/TNSM.2025.3596692

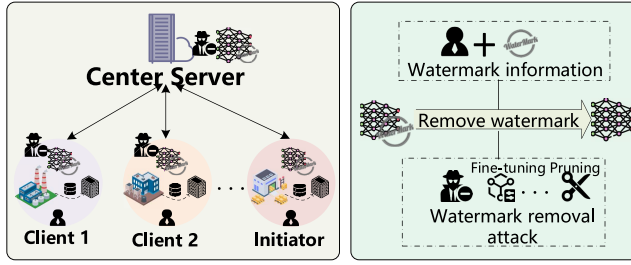


Fig. 1. Federated model leakage scenario and watermark removal methods.

or client access to the watermark embedding information. This pattern is capable of adapting the secure federated learning framework based on homomorphic encryption. This paper focuses on describing the watermark training process in this secure federated learning framework, as detailed in Section V. More importantly, we design an innovative approach to watermark training and robustness verification. This approach employs a secret neural network, with its parameters divided into a watermark and a watermark key. The watermark is embedded in the FL model, and the watermark key is used for ownership verification. In the validation phase, we design a watermark recovery method to counter complex watermark removal attacks, thereby enhancing the robustness of the watermarking technique.

Fed-RWM is rigorously evaluated using four popular DNN models (including CNN and RNN) and four benchmark datasets. It demonstrates advanced performance in the following metrics. In terms of fidelity, the accuracy of the FL watermarking model is nearly identical to that of the no-watermarking model, with a difference of less than 1%. In terms of validity, it can embed watermarks on both global and local models, and the watermark validation performance can reach 95%-98%. In terms of robustness, Fed-RWM is able to robustly cope with parameter scaling, hybrid novel attacks, and maintains its ownership verification capability without significant degradation. Furthermore, our method can embed watermarks across various network architectures with generality.

In summary, the main contributions of this paper are as follows:

- 1) We propose Fed-RWM, a robust watermarking method for FL models. The method trains a neural network model, sharing some parameters as watermarks with the FL model, and using the remaining parameters as keys for watermark verification. To the best of our knowledge, this is a novel approach, distinct from existing methods, and the first FL model watermarking method capable of preventing both server-side and client-side dishonest behavior.
- 2) We design a novel and robust ownership verification method. It combines watermarking and watermarking keys within a new secret network model, leveraging the model's performance on secret data as an indicator for ownership verification. Additionally, we integrate a watermark recovery function to counter novel watermark elimination attacks, enhancing the robustness of the verification process.

- 3) We conduct experiments on Fed-RWM across multiple models and datasets, including CIFAR-10, MNIST, IMDB, and WUSTLIOT-2021, to demonstrate its applicability in image, text, and IOT network intrusion detection tasks. We also consider non-independent identically distributed (non-IID) and independent identically distributed (IID) scenarios, the results show that Fed-RWM performs well in terms of robustness, validity, fidelity, and local model protection, etc.

The remainder of the paper is as follows: Section II reviews recent research on DNN and FL model watermarking. Section III introduces the preliminaries of FL and FL based on homomorphic encryption. Section IV introduces the threat model. Section V explains the composition and workflow of Fed-RWM. Section VI presents the experimental results. Finally, Section VII provides the conclusion.

II. RELATED WORK

In this section, we first introduce the related works on DNN watermark, and then review the latest development of FL-DNN watermark methods and discuss their shortcomings.

A. DNN Watermark

Deep neural network watermarking is an effective way to prevent the unauthorized use of models. This method embeds a valid watermark into the model's parameters or functions, and verifies ownership through a specific validation mechanism, which prevents plagiarism [12]. Currently, DNN watermarking methods in academia are primarily categorized as either white-box and black-box methods.

1) *White-Box Watermarking*: White-box watermarking mainly embeds the watermark by modifying the model's structure or parameters, requiring access to its internal architecture and parameter space [12]. During validation, it verifies ownership by extracting the model watermark and comparing it to the original watermark.

Uchida et al. [11] proposed the first white-box method, employing the parametric regularizer to embed an N -bit string watermark s ($s \in \{0, 1\}^N$) into the convolutional layer weights during protected model training. They extract watermark $\hat{s}$ from the suspicious model M_s , then calculate the hamming distance between $\hat{s}$ and s to verify ownership. This method demonstrates fidelity and verifiability and is robust to fine-tuning and pruning attacks.

Fan et al. [13] proposed a passport-based DNN ownership verification scheme, which adds a passport layer after the convolutional layer. They embed the watermark into the scale parameters of the BN layers. During verification, they add the passport layer to the network and detect the watermark to verify the identity. Chen et al. [14] embed model fingerprint (watermark) into the probability density function of trainable weights. They extract the fingerprint and compute the dot product with the owner's signature to verify models. Nevertheless, most white-box methods focus on CNN-based models and do not consider the damage of the parameter scaling attack.

Unlike the above methods, Lv et al. [15] proposed the HufuNet, which trains an encoder-decoder network, embeds

TABLE I
CATEGORIZATION OF EXISTING FL WATERMARKING METHODS B: BLACK-BOX, W: WHITE-BOX, S: EMBEDDING ON THE SERVER, C: EMBEDDING ON THE CLIENT, I: EMBEDDING ON THE INITIATOR

Methods	Embedding strategy	Attack resistance	Verification mechanism	Categories
Shao et al. [8]	Black-box (trigger set) in global model (server), white-box (weight matrix) in BN layers of local model (client).	Fine-tuning, overwriting, backdoor mitigation, pruning.	Trigger set and similarity score.	BS, WC
Chen et al. [9]	Generating adversarial samples to extract global model features as watermark (server).	Fine-tuning, adaptive, copevaatt, pruning.	Detecting adversarial samples in the output of suspicious models.	BS
Li et al. [10]	Black-box (trigger set) in global model (client), white-box (weight matrix) in BN layers of global model (client).	Fine-tuning, pruning, triggers against adversary.	Trigger set and hamming distance.	BC, WC
Yang et al. [21]	Black-box (trigger set + scaling factor) in local model (initiator).	Fine-tuning, pruning, PST.	Trigger set.	BI
Atli et al. [22]	Black-box (trigger set) in global model (server).	Fine-tuning, pruning, neural cleanse.	Trigger set.	BS
Chen et al. [23]	Black-box (trigger set) in hierarchical local model (client), white-box (weight matrix) in local model (client).	Fine-tuning, pruning.	Trigger set and hamming distance.	BC, WC
Wu et al. [24]	Black-box (trigger set) in local model (client), white-box (weight matrix) in global model (server).	Fine-tuning, pruning, clipping.	Trigger set and hamming distance.	BC, WS
Our	White-box (parameters sharing) in global model (initiator).	Fine-tuning, pruning, parameter scaling, overwrite, hybrid.	Performance of secret network on secret dataset (with recovery mechanism).	WI

the encoder as the watermark in the DNN model and utilizes the decoder for verification. It is capable of protecting models based on CNN and RNN architectures. Inspired by this approach, we develop a secret network composed of convolutional and fully connected layers, embedding half of its parameters in every layer of the protected model and verifying the model through the remaining parameters. We apply the designed method to model protection in federated learning.

2) *Black-Box Watermarking*: Black-box watermarking employs backdoor attack techniques, constructs key samples, and includes them as triggers in the training set [16]. This method embeds the backdoor watermark in the DNN model by simultaneously learning primary and backdoor tasks. During validation, the trigger set is input into the model, activating the backdoor task to verify ownership. Importantly, it does not require access to the model's internal architecture.

Bansal et al. [17] proposed a stochastic smoothing technique that ensures the parameters of the backdoor watermarking model fluctuate within an l_2 normative threshold, minimizing the effect of the watermarking task. This method enhances robustness against backdoor watermark removal attacks. Lao et al. [18] modified small-scale parameters in the protected model, ensuring it outputs pre-specified labels when inputting key samples, serving as proof of ownership. Other than that, there are other similar black-box watermarking approaches [19], [20]. Despite these innovations, the adversaries can impersonate the legitimate owner by searching for adversarial samples and using them to construct trigger sets [15].

B. FL-DNN Watermark

In federated learning, some researchers have proposed various FL model watermarking methods, which we classify according to the location and embedding method of the watermark, see Table I.

1) *Client-Side Watermarking*: Yang et al. [21] proposed a client-side backdoor watermarking method applied to secure FL. It places key samples with the FL initiator and increases their contribution through a scaling factor, thus embedding the backdoor watermark into the FL model. However, it compromises fairness in FL systems. Li et al. [10] proposed FedIPR, allowing clients to generate triggers and features to embed watermarks during local training through black-box and white-box techniques. However, due to the existence of semi-honest clients, FedIPR cannot ensure the embedding of correct watermarks and cannot resist novel watermark removal attacks.

2) *Server-Side Watermarking*: Tekgul et al. [22] introduced WAFFLE, a server-side black-box watermarking method, which embeds backdoor watermarking by retraining the global model. It is the first approach designed to protect FL models. Chen et al. [9] proposed FedRight, a black-box watermarking method. It generates adversarial samples from key samples as model fingerprints. These fingerprints are fed into the global model, with the output distribution features recorded and used as training data for a detector. During verification, the fingerprints are input into the suspicious model, and the detector analyzes its output features to determine ownership.

Wu et al. [24] proposed the Verifiable Property Federated Learning framework (VPFL) applied to the IoT, which embeds white-box watermarking on the server-side for the global model. Shao et al. [8] developed FedTracker, embeds black-box watermarks in the global model and distinct white-box watermarks for every client in the BN layer, which has traceability. Chen et al. [23] designed FedMCT, which considers the unbalanced resources of heterogeneous clients and groups them. In this method, the local model of each grouping has the same black-box watermark, and different white-box watermarks, which improves the tracking efficiency. However, these methods are not resistant to novel attacks and ignore client dishonesty.

In summary, existing server-side and client-side approaches fail to account for scenarios where both parties are dishonest. Although [21] addresses the issue, it expands the initiator's contribution, which is obviously not fair. Since black-box watermarking can be deciphered through input-output analysis, existing white-box methods rely on [11], which fail to resist novel attacks, thus we propose a novel white-box watermarking technique, distinct from existing methods. Our method resists novel attacks and provides enhanced security, as the watermarking information is known only to the initiator. Furthermore, it can be adapted to secure federated learning scenarios based on homomorphic encryption and models with diverse network architectures.

III. PRELIMINARIES

A. Federated Learning (FL)

FL is a distributed machine learning framework allowing multiple participants to train a global model collaboratively without sharing their data. It is an excellent solution to the issue of data lack and privacy preservation. A typical FL system consists of n clients $C = \{C_1, \dots, C_i, \dots, C_n\}$ and a central server S . In every round $r \in \{1, 2, \dots, R\}$, client C_i downloads the global model G^{r-1} from S . Afterwards, the parameters w_g^{r-1} of G^{r-1} are used to initialize the local model L_i^{r-1} with parameters w_i^{r-1} . After iteratively training (1) using local data $D = \{D_1, \dots, D_i, \dots, D_n\}$, C_i uploads w_i^r to S . S performs the aggregation calculation on the collected $\{w_1^r, \dots, w_i^r, \dots, w_n^r\}$ to obtain w_g^r and sends them to clients.

$$w_i^r = w_i^{r-1} - \rho \nabla L(w_i^{r-1}; D_i) \quad (1)$$

$$w_g^r = \sum_{i=1}^n \lambda_i^r w_i^r \quad (2)$$

where i is the index of clients, r is the index of training round, ρ is the learning rate. $L(\cdot; \cdot)$ is loss function, w_i^r is the local model parameters of client C_i in r th round. λ_i^r represents the aggregation weight, w_g^r represents the global model parameters in r th round.

The optimization objective of FL is outlined in (3). Through multiple training rounds, FL will end when the global model converges or reaches the defined training rounds.

$$\arg \min_{w_g \in \mathbb{R}^d} l_g(w_g) = \sum_{i=1}^n \lambda_i l_i(w_g), \quad l_i(w_g) = \mathbb{E}_{x_j \sim D_i} L(w_g, x_j) \quad (3)$$

where l_i represents the local optimisation objective, l_g represents the global optimisation objective, x_j represents the sample in D_i .

B. Federated Learning Based on Homomorphic Encryption

Researchers have confirmed that FL is only partially secure. Semi-honest servers can recover local private data from model gradient parameters [5]. Therefore, they have integrated privacy-preserving techniques like differential privacy [25],

homomorphic encryption [26], and secure multi-party computation [27], resulting in secure federated learning (SFL). In this paper, we focus primarily on HE.

Specifically, the HE algorithm generates a pair of public and private keys (pu_key , pr_key) for clients. The client C_i encrypts w_i^r using pu_key to obtain Enw_i^r and then uploads it. S directly aggregates the collected ciphertexts $\{Enw_1^r, \dots, Enw_i^r, \dots, Enw_n^r\}$ to obtain encrypted global model parameters Enw_g^r . C_i decrypts Enw_g^r using the pr_key . Through the encryption process, the server is unable to access the plaintext parameters, effectively preventing its dishonest actions.

IV. THREAT MODEL

In this paper, we assume that both the server and the client are dishonest, and both could potentially use some methods to remove the watermark and thus steal the model.

1) *Adversary's Objective*: They aim to remove the watermark without significantly compromising the performance of the stolen model. More clearly, their objectives are formulated as follows:

$$\begin{cases} f_v(w) & = True \\ \arg \max P(y_{ij}|x_{ij}; w) \approx \arg \max P(y_{ij}|x_{ij}; \hat{w}) & \text{s.t. } \forall x_{ij} \in X, \forall y_{ij} \in Y \end{cases} \quad (4)$$

where $f_v(\cdot)$ denotes the verification procedure, w denotes the stolen watermarked model parameters, $\hat{w}$ denotes the watermarked model parameters after removal attacks, x_{ij} and y_{ij} are the j th sample and label in D_i , X and Y are the sample space and label space of D .

2) *Adversary's Knowledge*: They possess the stolen FL model, fully understand its parameters and architecture, and are proficient in deep learning principles and model watermarking techniques.

3) *Adversary's Capability*: They can access local datasets, high-performance local and global watermarked models, and attempt to remove watermarks using the following deep learning techniques:

Fine-tuning. The fine-tuning attack is an adversary retraining the watermarked model on an additional dataset using a low learning rate [28], [29]. Owing to the non-convex nature of deep learning [30], the attack drives the model into a different local optimum, resulting in altered model parameter distributions and thus removing the watermark.

Pruning. The pruning aims to reduce model complexity by eliminating unimportant parameters [31]. As pruning may eliminate watermark parameters, it also can be considered a threat. In this paper, we implement the pruning attack using the method described in [11]. Specifically, we rank the parameters of the watermarked model in every layer by absolute value and sequentially set them to zero according to the pruning rate.

Parameter scaling. The parameter scaling attack targets neural networks with *relu* activation functions [32]. Specifically, it scales up the parameters of one layer and scales down the parameters of the next layer by a factor of m . The modified neural network's performance remains nearly unchanged, but the altered parameters may compromise the watermark. Therefore, it also poses a threat to the watermark.

TABLE II
SYMBOLS AND EXPLAINS

Symbols	Explains
w_m	The watermark
w_k	The watermark key
w_s^r	The model parameters of SNet in round r
D_s	The dataset of SNet
D_{st}	The test set of SNet
sp	The position of the watermark in the protected model
rp	The one-dimensional position of the watermark
M_s	The suspicious model
M	The watermarked model
M_u	The unprotected model
w_g^r	The global model parameters in round r
C_i	The client C_i
D_i	The dataset of client C_i
w_i^r	The local model parameters of client i in round r
Enw_i^r	The encrypted w_i^r
pu_key	The public key
pr_key	The private key
Θ_s^l	Watermarking model parameter matrix for layer l
Θ_s^l	Suspicious model parameter matrix for layer l
sf^l	Scaling factor of layer l
sd^l	Scaling factor direction of layer l
sv^l	Scaling factor value of layer l
$w_{recovery}$	Suspected mode parameters after recovery

Hybrid. The hybrid attacks combine three watermark removal attacks: fine-tuning, pruning, and scaling. Sophisticated adversaries may execute these attacks simultaneously to eliminate watermarks.

Overwrite. The overwrite attack involves an adversary embedding their watermark into the model using the same approach, which may result in the original watermark being overwritten or modified.

V. THE PROPOSED METHOD

A. The Components of Fed-RWM

In this paper, we propose Fed-RWM, an approach designed to protect the ownership of FL model in the IoT. Fed-RWM comprises three components: an initiator, multiple clients, and a server. Their capabilities are elaborated as follows.

Initiator. The initiator $I (I \in C)$ represents an industrial agent or smart device that initiates federated learning tasks in IoT. It can participate in local model training as a client. In addition, it undertakes tasks such as watermark generation, embedding & training, and ownership verification.

Clients. Client groups C represent industrial agents or smart devices participating in federated learning. They are mainly responsible for decrypting global model, training and encrypting local models.

Server. The server S represents the cloud server, which performs homomorphic encryption aggregation on the uploaded local model, and randomly selecting clients to participate in the training.

B. The Workflow of Fed-RWM

The main workflow of Fed-RWM is illustrated in Fig. 2 and Alg. 1. The main symbols are presented in Table II.

Algorithm 1 Workflow of Fed-RWM

Input: Total number of clients n , Percentage of training clients k , Total rounds R , Local learning rate ρ_l , Local epoch E_l , Local dataset D_i , Encryption and decryption keys pu_key , pr_key .

Output: Protected global model parameters. w_g^R

Initiator:

- 1: Initialize the parameters of protected model and SNet: w_g^0 , w_s^0 ;
- 2: $w_g^0 \xrightarrow[pu_key]{En(\cdot)} Enw_g^0$; // Encrypted upload to server
- 3: $w_s, sp = WatermarkGeneration()$; // Alg. 2

Server:

- 4: **for** $r = 1$ to R **do**
- 5: Select nk clients as the training subset C^r ;
- 6: **for** i in C^r **do**
- 7: $Enw_i^r = LocalUpdate(Enw_i^{r-1})$;
- 8: **end for**
- 9: $Enw_g^r = \sum_{i \in C^r} \frac{|D_i|}{\sum_{i \in C^r} |D_i|} \times Enw_i^r$;
- 10: $Enw_g^r = Embedding_Training(Enw_g^r)$; // Alg. 3, execute in the Initiator
- 11: **end for**
- 12: **return** $Enw_g^R \xrightarrow[pr_key]{De(\cdot)} w_g^R$.

Client:

Procedure $LocalUpdate()$

- 13: $Enw_g^{r-1} \xrightarrow[pr_key]{De(\cdot)} w_g^{r-1}$;
- 14: $w_i^{r-1} \leftarrow w_g^{r-1}$;
- 15: **for** epoch $e = 1, 2, \dots, E_l$ **do**
- 16: **for** each batch $b = \{x, y\} \in D_i$ **do**
- 17: $w_i^r \leftarrow w_i^{r-1} - \rho_l \nabla L(w_i^{r-1}; b)$;
- 18: **end for**
- 19: **end for**
- 20: $w_i^r \xrightarrow[pu_key]{En(\cdot)} Enw_i^r$;
- 21: **return** Enw_i^r .

Specifically, we create a secret network model (SNet), embed some of its parameters w_m into the protected FL model M , and utilize the remaining parameters w_k as the watermarking key for verification. The combination of w_m and w_k constitutes the SNet's complete parameter space w_s . We utilize w_s to protect M and to identify suspicious models. The implementation details of watermark generation, watermark embedding & training, and ownership verification are provided in Sections V-C–V-E. Next, we elaborate on Fed-RWM:

1) *Initialization:* The initiator I issues the FL task to the clients and the server. Specifically, I first creates the protected FL model M and initializes its parameters w_g^0 . Afterwards, it encrypts and uploads $Enw_g^0 = En(w_g^0)$ to server S . Here, $En(\cdot)$ represents the encryption computation. Finally, I constructs the secret network SNet to handle the watermarking task and initializes its parameters to w_s^0 .

After receiving the FL task, S counts the participating client group C , receives Enw_g^0 from I , and randomly selects a subset C^r from C to participate in the training round.

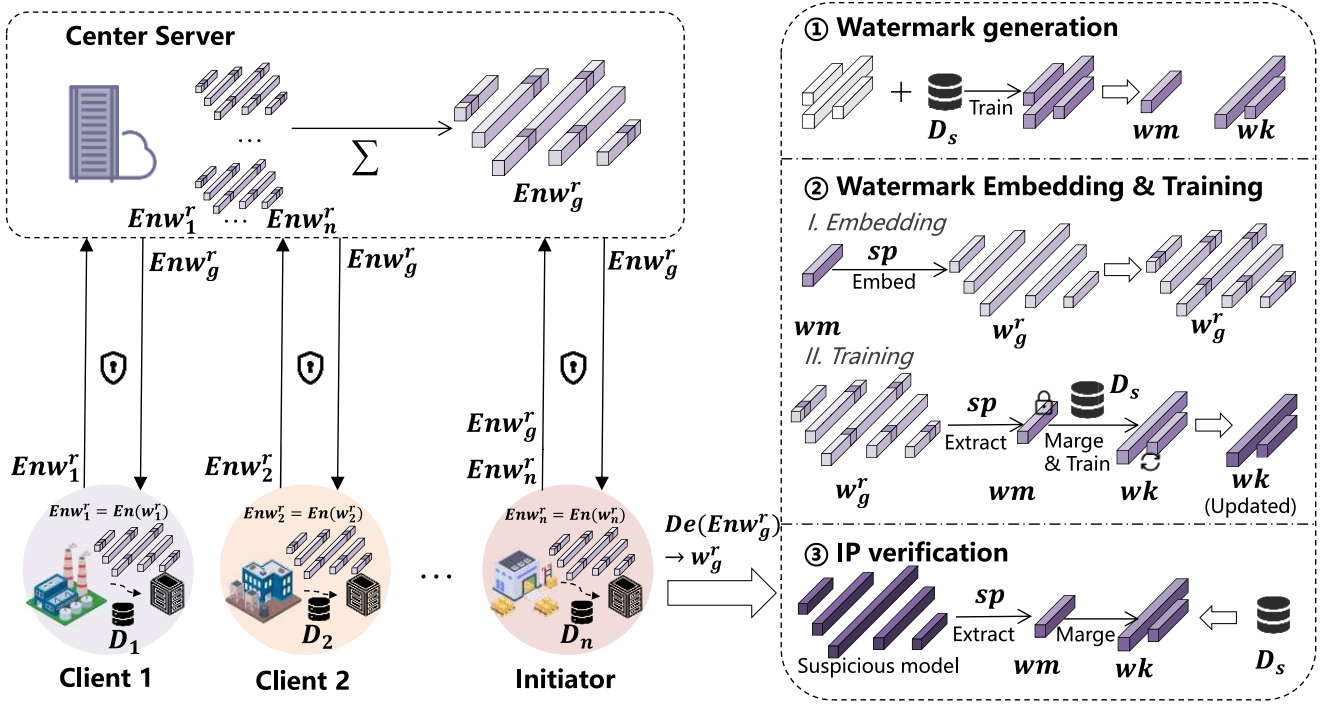


Fig. 2. Workflow of Fed-RWM.

These clients in C are required to create local models $L = \{L_1, \dots, L_i, \dots, L_n\}$ locally based on M .

2) *Initiator Generates Watermark:* The initiator completes the watermark generation task at the first FL training round, thus obtaining the initial watermark wm^r and the initial watermark key wk^r (i.e., $r = 1$). For detailed information, see Section V-C.

3) *Client Local Training:* The client C_i downloads and decrypts Enw_g^{r-1} ($De(Enw_g^{r-1}) = w_g^{r-1}$, $De(\cdot)$ is decryption operation) from S , using it to initialize w_i^{r-1} . Subsequent, C_i initiates the *LocalUpdate* procedure, which uses local data D_i to train w_i^{r-1} via gradient descent (1). Finally, the client encrypts w_i^r and uploads it to the server. Notably, the initiator can also participate in training as a client if it is selected into the training subset.

4) *Server Aggregation:* S aggregates the collected local model ciphertexts to obtain Enw_g^r and sends it to the initiator. Later, it instructs the initiator to start the *Embedding_Training* procedure for embedding or training the watermark and receives Enw_g^r back from the initiator. Additionally, S is also required to select the client subset C^{r+1} for the next round randomly.

5) *Initiator Embeds and Trains Watermark:* I receives and decrypts Enw_g^r from the server. It then starts the *Embedding_Training* procedure, which procedure embeds the watermark wm in the first round and trains SNet to update the watermark key wk in the remaining rounds. In every round of FL, the current wm and wk are saved locally, and the watermarked w_g^r is then returned to the server after encryption. We will describe this procedure in detail in Section V-D.

6) *End of Training:* Continuously repeat steps 3-5 until either the number of training rounds R is reached or convergence is achieved.

C. Watermark Generation

The watermark generation procedure is implemented in the initiator and consists of two parts: watermark generation and position generation. Its workflow is shown in Alg. 2 and Fig. 2.

1) *Watermark Generation:* SNet is composed of four convolutional layers and one fully connected layer. We define the first two convolutional layer parameters as wm and the remaining layer parameters as wk . Here, $wm \cup wk = w_s$ and $wm \cap wk = \emptyset$. wm and wk are jointly utilized to extract the features of secret data, with their classification performance serving as a criterion for verifying ownership. When the watermark generation process begins, the SNet is trained on the small secret dataset D_s with a few iteration rounds. It is important to note that D_s and FL task's dataset can be entirely different. After training, wm is used as the watermark to embed in the first round of FL, while wk serves as the corresponding watermark key. Both are stored in the initiator.

2) *Positions Generation:* About the position of the watermark embedding, we generate it using random sampling. Random generation makes the position information of the watermark not inferable, and it is difficult for the adversary to detect the position information by analysing the watermark parameters and model parameters. Specifically, we count the number of all parameters of the protected model M except the *softmax* layer, denoted as $|w_g|$. After that, we count the number of parameters in wm , denoted as $|wml|$. Next, we randomly generate $|wml|$ values in the range $[0, |w_g|]$ as one-dimensional watermark positions (5). Since the network structure of M is mostly multi-dimensional, we convert rp into the corresponding multi-dimensional positions sp , as described in Alg. 2. Each element in sp is a mask matrix consistent with the shape of w_g (i.e., the value is 1 at selected positions and

Algorithm 2 WatermarkGeneration

Input: M 's parameters w_g , SNet learning rate ρ_s , SNet epoch E_s , Secret dataset D_s .

Output: SNet parameters w_s , secret position sp .

Positions Generation:

- 1: $rp \leftarrow \text{Eq. 5}$;
- 2: $layers \leftarrow M$'s total number of layers except *softmax* layer;
- 3: $sp = \{0 \cdot w_g\}^{|rp|}$;
- 4: **for** $i = 0$ to $|rp|$ **do**
- 5: **for** $ls \leq layers$ **do**
- 6: $N_{ls} \leftarrow$ Number of parameters in layer ls ;
- 7: **if** $rp[i] \leq N_{ls}$ **then**
- 8: Set the value of this position in sp_i to 1;
- 9: **else**
- 10: $rp[i] - = N_{ls}$;
- 11: **end if**
- 12: **end for**
- 13: **end for**

Watermark Generation:

- 14: **for** epoch $e = 1, 2, \dots, E_s$ **do**
- 15: **for** each batch $b = \{x, y\} \in D_s$ **do**
- 16: $w_s \leftarrow w_s - \rho_s \nabla L(w_s; b)$;
- 17: **end for**
- 18: **end for**
- 19: **return** w_s, sp .

0 for the rest). The embedding positions defined by sp cover nearly every layer of M , allowing us to incorporate watermark information across most of them. In contrast to state-of-the-art (SOTA) FL white-box methods [8], [10], [23], which embed watermarks into BN layers, our approach is more versatile and can be applied even to smaller models that lack BN layers.

$$rp = \{rp_0, \dots, rp_i, \dots, rp_j, \dots, rp_{|wm|}\} \subseteq [0, |w_g|]$$

$$\text{s.t. } i \neq j, \quad rp_i \neq rp_j \quad (5)$$

where w_g denotes the global model parameters of M , i and j denote the index of rp .

D. Watermark Embedding and Training

The watermark embedding and training in our approach takes place in the initiator. During the first round of FL (i.e., $r = 1$), the initiator receives Enw_g^r from the server and decrypts it into w_g^r . Next, we sequentially embed the watermark w_m generated from Alg. 2 into w_g^r according to the position sp . This embedding procedure will result in watermarked FL model parameters w_g^r . These parameters are encrypted and sent to the server as the FL global model parameters w_g^r for this round. Notably, in the first round, the model has yet to converge, and the parameters float is vast, which can effectively ensure the concealment of the embedded watermark.

In subsequent training rounds, we carry out the following steps (6)–(9). First, the initiator extracts wm^r from w_g^r using the predefined position (6). Then, we merge wm^r with wk^{r-1} which from the previous round as the parameters w_s^{r-1} for

Algorithm 3 Embedding_Training

Input: $\rho_s, E_s, D_s, pu_key, pr_key, Enw_g^r, sp, wk^{r-1}$.

Output: Protected encryption global model parameters Enw_g^r .

- 1: $Enw_g^r \xrightarrow[pr_key]{De(\cdot)} w_g^r$;
- 2: **if** $r=1$ **then**
- 3: $w_s^r \xrightarrow{divide} [wm^r, wk^r]$; // w_s^r from Alg. 2.
- 4: $w_g^r \leftarrow \text{Embedding}(sp, wm^r, w_g^r)$; //Based on sp, wm^r are sequentially embedded in w_g^r .
- 5: **else**
- 6: $wm^r \leftarrow \text{Extract}(sp, w_g^r)$; //Extract the watermark in w_g^r .
- 7: $w_s^{r-1} = wm^r \cup wk^{r-1}$;
- 8: **for** epoch $e = 1, 2, \dots, E_s$ **do**
- 9: $w_s^r \leftarrow \text{Eq. 8}$;
- 10: **end for**
- 11: $wk^r \leftarrow \text{Eq. 9}$;
- 12: **end if**
- 13: Store wm^r, wk^r ;
- 14: **return** $Enw_g^r \xleftarrow[pu_key]{En(\cdot)} w_g^r$.

SNet (7). Due to the federated forgetting phenomenon, w_g^r changes in every round of FL, which means wm^r may not match wk^{r-1} . This mismatch could result in w_s^{r-1} underperforming compared to the previous round, thereby hindering watermark verification. To avoid this, we continuously fine-tune w_s^{r-1} at each round of communication based on continuous learning, resulting in w_s^r . Unlike the training process in the watermark generation phase, we freeze the parameter space of wm^r and update wk^{r-1} to wk^r by gradient descent (8). In order to prevent the fine-tuning from leading to SNet overfitting, we set up an early stopping mechanism. This mechanism saves the model parameters from the previous round if performance degrades on the validation set. After fine-tuning, wk^r is able to adapt to wm^r and w_s^r will regain its original performance (9).

$$wm^r = \text{Extract}(sp, w_g^r) = \begin{pmatrix} \sum sp_0 \odot w_g^r \\ \vdots \\ \sum sp_{|sp|} \odot w_g^r \end{pmatrix} \quad (6)$$

$$w_s^{r-1} = wm^r \cup wk^{r-1} \quad (7)$$

$$w_s^r = w_s^{r-1} - \rho_s \nabla_{wk^{r-1}} L(wm^r, wk^{r-1}; D_s) \quad (8)$$

$$wk^r = w_s^r \setminus wm^r \quad (9)$$

where $\text{Extract}(\cdot)$ denotes extract model watermark, wm^r denotes wm in r th round, w_s^r denotes the full parameters of the SNet model in the r th round.

Through the simultaneous and coordinated learning of the watermarked FL model and SNet, the FL model achieves higher performance on the primary task when SNet also performs well on D_s . Consequently, the optimization objective of Fed-RWM can be formulated as shown in (10). It is important to note that $|wm| \ll |w_g|$, meaning the number of parameters in wm is significantly smaller than in w_g ,

minimizing the negative impact of the embedding and training of the watermark on the protected FL model.

$$\begin{aligned} \arg \min_{w_g \in \mathbb{R}^d} l(w_g) &= \sum_{i=1}^n \lambda_i L(w_g; D_i), \arg \min_{w_s \in \mathbb{R}^d} l(w_s) = L(w_s; D_s) \\ \text{s.t. } w_s &= wm \cup wk, wm \subset w_g \end{aligned} \quad (10)$$

E. Ownership Verification

When participants in FL discover the suspicious model, the initiator I is notified immediately. I triggers the verification procedure to verify the model property rights. In this paper, we set up a test dataset D_{st} in the initiator with the same distribution as D_s but without overlapping samples. For clarity, we notate the watermark extracted from M_s as w_{m_s} , and the merged w_{m_s} and w_k as $\hat{w}_s$. We set a validation metric, model accuracy score AS .

$$AS = \frac{1}{|D_{st}|} \sum_{(x,y) \in D_{st}} \mathbb{I}(\arg \max f_{\hat{w}_s}(x) = y) \quad (11)$$

where $\mathbb{I}(\cdot)$ denotes the indicator function, $f_{\hat{w}_s}(\cdot)$ denotes the prediction of test sample x with $\hat{w}_s$. AS represents the accuracy of $\hat{w}_s$ on D_{st} . The larger its value, the more watermarking information the protected model contains. Due to the non-convexity of the deep model, it is almost impossible for an innocent model to perform well on AS . On the contrary, infringing models have higher values on AS . We adopt a hard threshold β to verify ownership, i.e., if $AS \geq \beta$, the suspected model is confirmed to be infringement.

Nevertheless, attackers may utilize watermark removal techniques to erase the watermarks, causing AS to fail to identify infringement on suspicious models following specific attacks. To enhance the robustness of ownership verification, this paper designs an enhanced verification method based on watermark recovery. Specifically, we perform AS validation on M_s . If the validation is successful (i.e., plagiarism is confirmed), we end the validation. If the behaviour does not exist, we perform the watermark recovery method prior to AS validation to confirm whether the model suffers from the scaling attack and hybrid attacks, respectively (Other attacks do not significantly affect the AS performance). The key point of the watermark recovery is to determine the scaling factor for each layer of M_s . For more, see Fig. 6, where the specific recovery steps are described below.

1) *Pruning*: Pruning and fine-tuning attacks can affect the accurate calculation of the scaling factor. To address this, we prune the parameters of each layer in both the watermarked and suspicious models prior to calculating the scaling factor. Our goal is to determine the scaling factor sf , and the pruning rate is designed without considering the impact on the model performance.

2) *2D Matrix Mapping*: We map the multidimensional parameters of each layer from both the suspicious and watermarking models into two-dimensional matrices Θ_s^l and Θ_g^l .

3) *Calculation of Scaling Factor Values*: For Θ_s^l and Θ_g^l , we perform singular value decomposition (SVD) to obtain the diagonal matrices Σ_s^l and Σ_g^l (12) respectively. Next, we calculate the traces of the two diagonal matrices, compute their

mean. These means are divided to obtain the scaled values sv^l (13) for each layer of the model.

$$\begin{aligned} \Theta_s^l &= U_s^l \Sigma_s^l V_s^{l*} \\ \Theta_g^l &= U_g^l \Sigma_g^l V_g^{l*} \end{aligned} \quad (12)$$

$$sv^l = \frac{\text{tr}(\Sigma_s^l)}{\text{tr}(\Sigma_g^l)} \quad (13)$$

where U and V^* are the left singular matrix and right singular matrix after matrix singular value decomposition. sv^l is the l -th layer scaling factor value. $\text{tr}(\cdot)$ is the trace of the matrix.

4) *Calculate the Scaling Factor Directions*: SVD cannot determine the direction difference between Θ_s^l and Θ_g^l . Thus we calculate their cosine similarity to reflect the direction sd^l . If the similarity is negative, the scaling factor of this layer is negative, i.e., sd^l is -1 , otherwise sd^l is 1 .

$$\begin{cases} \frac{\Sigma_s^l \cdot \Sigma_g^l}{\|\Sigma_s^l\| \cdot \|\Sigma_g^l\|} \leq 0 & sd^l = -1 \\ wm^{r-1} \frac{\Sigma_s^l \cdot \Sigma_g^l}{\|\Sigma_s^l\| \cdot \|\Sigma_g^l\|} > 0 & sd^l = 1 \end{cases} \quad (14)$$

5) *Watermark Recovery and Verification*: We multiply the sv^l and sd^l values for each layer to compute the scaling factor sf^l (15) of each layer. The combination of pruning and fine-tuning attacks causes the pruned parameters to deviate significantly from the original parameters after fine-tuning. These deviations are further amplified by the scaling factor, compromising the recovery of the model and watermark. Therefore, We eliminate these parameters by properly pruning the suspicious model before recovery. We utilize the highest pruning rate p that can maintain the performance of the model. The scaling factors are then applied to the corresponding layer parameters of the pruned suspicious model to reconstruct the original model (16)-(17). Finally, the watermark of the recovered model is extracted and combined with the watermark key to verify ownership.

$$sf^l = sv^l sd^l \quad (15)$$

$$w_{recovery} = \begin{pmatrix} sf^0 \\ \dots \\ sf^l \\ \dots \\ sf^{layers} \end{pmatrix} \odot \begin{pmatrix} \text{prune}(w_{sus}^0, p) \\ \dots \\ \text{prune}(w_{sus}^l, p) \\ \dots \\ \text{prune}(w_{sus}^{layers}, p) \end{pmatrix} \quad (16)$$

$$\begin{aligned} \text{prune}(w, p) &= w \odot m \\ \text{s.t. } m_{i,j} &= \begin{cases} 1 & |w_{ij}| \geq \tau(p) \\ 0 & |w_{ij}| < \tau(p) \end{cases} \end{aligned} \quad (17)$$

where $\text{prune}(\cdot)$ is pruning, p is pruning rate, m is a pruning mask matrix, $\tau(\cdot)$ is a function that determines the pruning thresholds, $layers$ is the total number of layers, $w_{recovery}$ are the parameters of recovered suspicious model M_s . Our validation method demonstrates robustness against both attacks after watermark restoration. In addition, we also effectively distinguish between innocent, plagiarized, and protected models.

VI. EVALUATION

A. Experimental Setup

1) *Experimental Environment*: All experiments are conducted using Python 3.6 and the Keras deep learning

library, running on an Ubuntu 16.04 server with dual Nvidia TITAN X GPUs.

2) *Datasets*: We conduct experiments on four datasets: CIFAR-10, MNIST, WUSTL-IIOT-2021 (IIOT2021) [33], and IMDB [34]. CIFAR-10, IMDB, and IIOT2021 serve as the primary datasets for FL tasks, while MNIST is used as the secret dataset D_s . For CIFAR-10, IMDB, and IIOT2021, we split their data into training and test sets with an 8:2 ratio. The training set serves as the data source for the FL tasks across all clients, while the test set is reserved for validation. It is partitioned into nk subsets based on IID and non-IID, allocated as local datasets for every client. Similarly, MNIST is divided into a training set for training SNet and a test set for ownership verification. IIOT2021 is an intrusion detection dataset in IIoT, comprising attack types such as denial of service, exfiltration, and reconnaissance. We employ it as an FL dataset to simulate a distributed intrusion detection system for industrial IoT to demonstrate the applicability of Fed-RWM in IoT.

3) *Models*: We conduct experiments on four models: ResNet18 [35], DenseNet121 [36], LSTM [37], and CNN. Specifically, we train ResNet18 and DenseNet121 in CIFAR-10, LSTM in IMDB, and CNN in IIOT2021.

4) *FL Setup*: For FL, we consider both data IID and non-IID scenarios and experimented with them both. The training loss function for both FL and SNet is cross-entropy loss, with Adam and SGD as the respective optimizers. Additional settings are provided in Table III.

5) *Evaluation Metric*: We utilize AS mentioned in Section V-E as the validation metric.

B. Validity

In this section, we evaluate the validity of Fed-RWM, focusing on two primary objectives: first, determine its ability to verify the ownership of protected models; second, evaluate its capability to distinguish between innocent and watermarked models. We conduct experiments between protected model M and unprotected model M_u (i.e., innocents). M_u is trained with the same training settings as M . To comprehensively evaluate the validity, we examine three distinct scenarios. ① Scenario 1, FL dataset differs from D_s , and M extracts the watermark at the same secret position with M_u . ② Scenario 2, FL dataset (i.e., MNIST) is identical to D_s , and M extracts the watermark at the same secret position with M_u . ③ Scenario 3, the FL dataset differs from D_s , and positions are different. Specifically, Scenario 1 represents the general case in reality. Scenario 2 shows that our approach is equally effective even if the dataset is the same. Scenario 3 illustrates that ownership can only be verified with correct position information. In addition to that, both IID and non-IID cases are considered. Regarding the design of hard thresholds in ownership verification, employing the empirical results, it is designed as $\beta = 0.6$. The comprehensive experimental results are presented in Table IV.

In Scenarios 1 and 2, the AS scores for the watermarks extracted from M exceed 95%, indicating that the models retain over 95% of the watermark information. In contrast, the experimental results for M_u show significant gaps, with

TABLE III
EXPERIMENTAL PARAMETER SETTING

Parameters	Descriptions	Value
ρ_l	FL learning rate	0.001
ρ_s	SNet learning rate	0.001
ρ_f	Fine-tuning learning rate	0.001/0.01
E_l	Local epoch	1
E_s	SNet epoch	100, early stopping
R	Federated learning rounds	50
n	Total number of clients	10
k	Percentage of selected clients	0.5

TABLE IV
THE VALIDITY OF PROTECTED FL MODEL (WATERMARKED MODEL) AND UNPROTECTED FL MODEL (INNOCENTS MODEL) IN THREE SCENARIOS

		Scenario.1	Scenario.2	Scenario.3
Protected	non-IID	ResNet18	0.9587	0.9773
		DesNet121	0.9689	0.9732
		LSTM	0.9877	0.9656
		CNN	0.9866	0.9774
		ResNet18	0.9584	0.9681
	IID	DesNet121	0.9550	0.9634
		LSTM	0.9872	0.9698
		CNN	0.9865	0.9756
	Unprotected	ResNet18	0.1088	0.0863
		DesNet121	0.1138	0.0792
Unprotected	non-IID	LSTM	0.1635	0.2462
		CNN	0.0890	0.0898
		ResNet18	0.1135	0.0994
	IID	DesNet121	0.1029	0.1402
		LSTM	0.0959	0.1158
		CNN	0.1273	0.1050

most M_u models achieving less than 20% in AS . This suggests that M_u contains minimal watermark information, indicating a lack of effective watermark protection. The reason for this is that the watermark key is updated under the secret dataset according to the watermark information in each round. The data extracted by the unprotected model under the position information does not match the watermark key and cannot perform well on the secret dataset. Therefore, based on the experimental data in Scenarios 1 and 2, our approach can effectively distinguish between watermark-protected and non-watermark-protected models (i.e., innocent or not), regardless of whether the FL dataset is the same as the secret dataset.

In Scenario 3, we generate the forged watermark position $\hat{sp}$ using the watermark generation algorithm. We then extract the parameters of M at $\hat{sp}$, and the corresponding experimental results are presented in Table IV. The results indicate that the AS values remain around 10%, falling short of the ownership verification threshold. This occurs because Fed-RWM cannot retrieve watermark information from an incorrect location, and such information cannot be matched with the corresponding secret key. The integrated secret network fails to leverage its full potential on the confidential dataset. This design prevents plagiarists from forging ownership by altering the position.

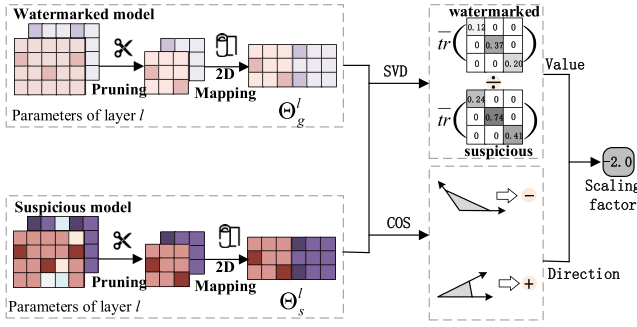


Fig. 3. Scaling factor calculation.

C. Fidelity

In this section, we assess the negative impact on the FL primary task. Fidelity demands that the watermark does not significantly degrade the performance of the primary task while verifying ownership. We compare the performance of four models on test sets from three different datasets, contrasting the outcomes of the Fed-RWM algorithm with those of the FedAvg algorithm [38]. It is worth noting that the FedAvg algorithm is the baseline federated learning approach without watermarking.

As illustrated by the results in Fig. 3, the watermarked model accuracies are 85.91%, 88.24%, 83.22%, 99.65% and 85.45%, 88.43%, 83.16%, 99.85% without watermark. In non-IID, the watermarked ones were 84.74%, 86.05%, 83.06%, 99.84%, and the non-watermarked ones were 84.25%, 86.73%, 83.14%, 99.84%. The watermarked models are only reduced by less than 1%. Our approach successfully embeds the watermark with negligible effect on the primary task of FL model. This is mainly because watermark parameters account for only a few model parameters. And, the watermark embedding scheme, which performs embedding once followed by multiple fine-tuning iterations, further mitigates adverse effects on the original model.

Other SOTA white-box FL model watermarking methods, FedTracker [8], FedIPR [10], and FedMCT [23], also perform well in fidelity. FedTracker and FedIPR report less than 2% impact on model performance, Fed-RWM incurs even less loss. FedMCT demonstrates a loss of less than 1%, which is comparable to our method. Unfortunately, it cannot adapt to complex watermark removal attacks.

D. Robustness

In Fed-RWM, only the initiator has permission to access the wm and sp , which are hidden from clients, the server, or plagiarizers. Thus, it is highly improbable that they will obtain the position and precisely eliminate the wm . Since the FL model is collaboratively trained by multiple clients, obtaining high-quality data from diverse sources to retrain the model is challenging for plagiarists. Additionally, plagiarists want to refrain from spending their computational and data resources to retrain the model. They prefer to modify the trained model through some attacks (e.g., fine-tuning, pruning and scaling), destroying the watermark. In this section, we reproduce attacks

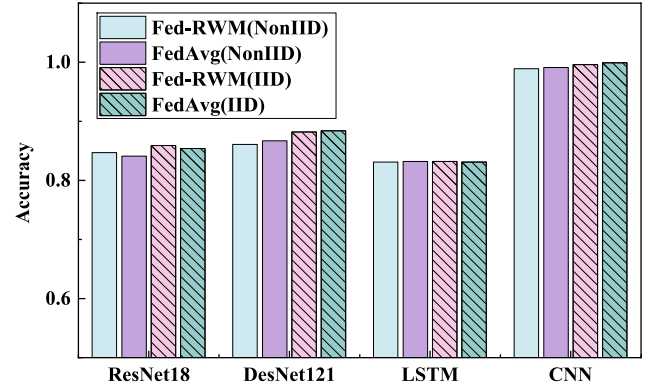


Fig. 4. The fidelity of Fed-RWM. Watermark model: Training the model using Fed-RWM. No watermark model: Training the model using FedAvg [38].

TABLE V
THE ROBUSTNESS TO FINE-TUNING ATTACK

Scenarios	ρ_f	Epochs	ResNet18	DesNet121	LSTM	CNN
non-IID	-	0	0.9587	0.9689	0.9877	0.9866
	0.001	100	0.9588	0.9689	0.9877	0.9866
	0.01	100	0.9588	0.9687	0.9878	0.9866
IID	-	0	0.9584	0.9550	0.9872	0.9865
	0.001	100	0.9578	0.9544	0.9872	0.9864
	0.01	100	0.9587	0.9546	0.9872	0.9864
Maximum loss			-0.06%	-0.06%	0%	-0.02%

described in Section IV to verify the robustness of our approach.

1) *Robustness Against Fine-Tuning Attack*: We consider the following two scenarios. Scenario 1, the fine-tuning dataset is the same as the FL dataset, representing the standard fine-tuning attack. Scenario 2, the fine-tuning dataset differs from the FL dataset, simulating a transfer learning-based fine-tuning attack. We train four models on three FL datasets (CIFAR-10, IMDB, and IIOT2021), and the experimental parameters are shown in Table III. These models all embed watermarks based on Fed-RWM.

In Scenario 1, following the setup of previous work [39], we assume that the adversary has access only to the FL dataset's test set. Consequently, all fine-tuning datasets are derived from the test sets of the three FL datasets. These datasets are divided into train and test subsets in an 8:2 ratio. These train subsets are used for model fine-tuning, while test subsets evaluate the effectiveness of the fine-tuning. The training settings are from [39], including $epochs = 100$, learning rates $\rho_f = 0.001$ or 0.01 , and the optimization method used is stochastic gradient descent. We compare the performance of the watermarked model before (i.e., $epochs = 0$) and after fine-tuning. The comparison shows that our approach experiences a maximum AS loss of only 0.06% after 100 epochs of fine-tuning. In some cases, AS values were slightly higher than before fine-tuning. These are due to the fine-tuning will cause the watermark parameters to be retrained and may enhance the performance of the SNet.

TABLE VI
THE ROBUSTNESS TO PRUNING ATTACK

Rate	ResNet18		DesNet121		LSTM		CNN	
	<i>Acc</i>	<i>AS</i>	<i>Acc</i>	<i>AS</i>	<i>Acc</i>	<i>AS</i>	<i>Acc</i>	<i>AS</i>
0.0	0.8474	0.9587	0.8605	0.9689	0.8306	0.9877	0.9985	0.9866
0.1	0.8464	0.9586	0.8561	0.9689	0.8306	0.9875	0.9985	0.9866
0.2	0.8467	0.9587	0.8564	0.9690	0.8306	0.9876	0.9986	0.9863
0.3	0.8455	0.9585	0.8539	0.9690	0.8319	0.9876	0.9985	0.9863
0.4	0.8403	0.9590	0.8469	0.9687	0.8293	0.9874	0.9980	0.9863
0.5	0.8298	0.9592	0.8302	0.9689	0.8275	0.9876	0.9972	0.9862
0.6	0.7909	0.9590	0.6305	0.9687	0.8206	0.9876	0.9960	0.9863
0.7	0.6516	0.9584	0.2307	0.9688	0.8046	0.9874	0.9911	0.9859
Loss	0.03%		0.01%		0.03%		0.07%	

In Scenario 2, unlike Scenario 1, we fine-tune the ResNet18 and DenseNet121 watermarked models using MNIST for transfer learning. Post-fine-tuning, the watermark detection performance scores *AS* for the two models are 0.9588 and 0.9687 in the non-IID setting, while they are 0.9573 and 0.9544 in IID, with a maximum performance loss of only 0.11% compared to pre-fine-tuning results.

All experimental results show that while fine-tuning slightly decreases *AS*, it remains consistently high. Fed-RWM is resilient to fine-tuning attacks.

2) *Robustness Against Pruning Attack*: We perform sparse pruning on the four watermarked models. Since IID and non-IID settings do not affect the watermarking function, we focus solely on non-IID scenarios. Following the method in [11], redundant parameters in every layer are pruned based on their absolute values. We evaluate the classification accuracy *Acc* of the watermarked models on the main task and the watermark verification performance *AS* across different pruning ratios. Table VI presents the results for *Acc*, *AS*, and *AS* loss at various pruning levels. As pruning increases, both *Acc* and *AS* decline. When pruning rate is 70%, *Acc* significantly drops, affecting the performance of the model. However, *AS* remains above 90%, the maximum loss is 0.07%, and ownership can still be verified based on validation conditions. Thus, Fed-RWM exhibits robustness against pruning attacks.

3) *Robustness Against Parameter Scaling Attack*: We also evaluate the robustness of Fed-RWM against the parameter scaling attack on ResNet18, DenseNet121, and CNN. Since this attack targets deep models with the *relu* activation function, we are not experimenting with the LSTM model, which uses *sigmoid* [32]. Parameter scaling on all convolutional layers may drastically affect the performance of complex models, which the attacker does not desire. Thus, we selectively scale the parameters of certain convolutional and BN layers in model M by a factor of 2^m ($m \in \mathbb{Z}$), with random scaling factors for different layers. We represent M after the attack as the suspect model M_s . The left side of table VII refers to the *Acc* and *AS* of M_s . Although the overall performance of M_s remains comparable to the original, the watermark is significantly compromised. *AS* drops below 60%, leading to the failure of watermark verification.

To enhance the robustness of ownership verification, we apply the watermark recovery method from Section V-E to recover M_s , followed by ownership verification on the

TABLE VII
THE ROBUSTNESS TO PARAMETER SCALING ATTACK

Model	Attacked		Recovered	
	<i>Acc</i>	<i>AS</i>	<i>Acc</i>	<i>AS</i>
ResNet18	0.8485	0.3015	0.8530	0.9587
DesNet121	0.8526	0.4148	0.8512	0.9689
CNN	0.9963	0.5032	0.9984	0.9866

TABLE VIII
THE ROBUSTNESS TO HYBRID ATTACKS

		ResNet18		DesNet121		CNN		Success rate
		<i>Acc</i>	<i>AS</i>	<i>Acc</i>	<i>AS</i>	<i>Acc</i>	<i>AS</i>	
PFS	Attacked	0.8415	0.3318	0.8915	0.4159	0.9969	0.5086	-
	Recovered	0.8750	0.9590	0.8710	0.9689	0.9981	0.9861	1.0
SPF	Attacked	0.8665	0.3317	0.8975	0.4157	0.9994	0.5093	-
	Recovered	0.8580	0.9592	0.8665	0.9686	0.9865	0.9854	1.0
FSP	Attacked	0.8355	0.3320	0.8505	0.4162	0.9944	0.4907	-
	Recovered	0.8630	0.9589	0.8515	0.9688	0.9981	0.9861	1.0
PSF	Attacked	0.8680	0.3317	0.8960	0.4157	0.9996	0.5096	-
	Recovered	0.8400	0.9589	0.8759	0.9668	0.9981	0.9854	1.0
SFP	Attacked	0.8635	0.3317	0.8769	0.4143	0.9964	0.5095	-
	Recovered	0.8435	0.9591	0.8635	0.9672	0.9967	0.9857	1.0
FPS	Attacked	0.8340	0.3324	0.8480	0.4162	0.9942	0.4912	-
	Recovered	0.8610	0.9591	0.8475	0.9687	0.9979	0.9861	1.0
Success rate		1.0		1.0		1.0		-

recovered model. The right side of Table VII presents the recovered *Acc* and *AS*. *Acc* recovers to 85.30%, 85.12%, and 99.84%, *AS* recovers to 95.87%, 96.89%, and 98.66%, which can recover to the performance before the attack, and thereby effectively verify the infringement of M_s . Hence, Fed-RWM can resist the parameter scaling attack.

4) *Robustness Against Hybrid Attacks*: Sophisticated adversaries could launch hybrid attacks that include fine-tuning (F), pruning (P), and parameter scaling (S) against the watermarked FL model M . In this section, we experiment with eight different sequences of hybrid attacks (i.e., PFS, SPF, FSP, PSF, SFP, FPS, FP, PF). Since the parameter scaling attack targets the *relu* activation function, PFS, SPF, FSP, PSF, SFP and FPS attack experiments are conducted solely on the ResNet18, DenseNet121, and CNN models, FP and PF attack experiments are conducted on the LSTM. In order to maximize the destruction of the model watermark, we chose the pruning rate to be 0.5 and the fine-tuning learning rate to be 0.001. As IID and non-IID distributions do not influence the experimental outcomes, we only experiment non-IID.

To ensure that Fed-RWM can still verify watermarks under hybrid attacks, we develop a recovery method to restore watermarks before ownership verification, see Section V-E.

Previous experimental results indicate that in the absence of attacks, ResNet18, DenseNet121, and CNN achieve accuracies of 0.8474, 0.8605, and 0.9985 for the main task, and exhibit ownership verification capabilities of 0.9587, 0.9689, and 0.9866, respectively. As shown in Table VIII, after hybrid attacks, the main task performance of the three models remain within the range of 0.8340~0.8680, 0.8480~0.8975, and 0.9942~0.9996, with no significant degradation. Due to retraining during the fine-tuning attack,

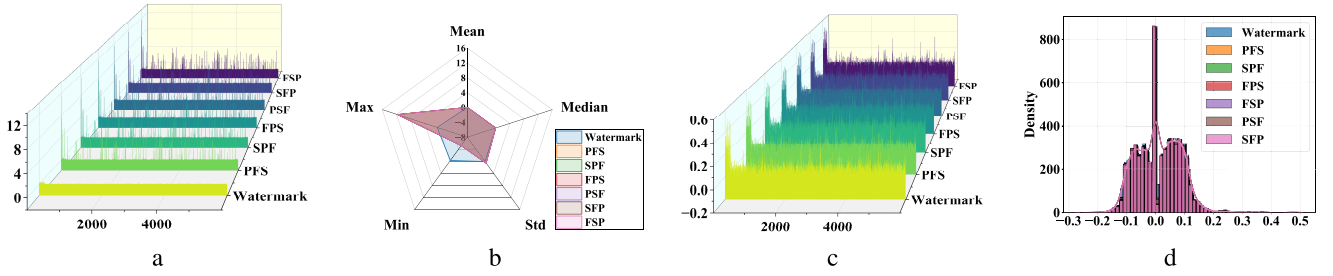


Fig. 5. Watermark distribution before attack, after attack and after recovery. (a. Distribution of watermark before and after hybrid attacks, b. Watermark statistics before and after hybrid attacks, c. Distribution of watermark after hybrid attacks and after recovery, d. Kernel density of watermark after hybrid attacks and after recovery).

Acc slightly improves following some hybrid attacks. The scaling attack severely damages the watermark, resulting in *AS* values of approximately 0.33, 0.42, and 0.51, which are insufficient for ownership verification. After recovery, the *AS* values improve to 0.9589~0.9592, 0.9668~0.9689, and 0.9854~0.9861. Compared to the pre-attack period, the maximum reduction is 1.31%. Although the *AS* values decrease after recovery, they are still sufficient to verify ownership. The performance of the recovered model may be degraded due to miscalculation of scale factors and changes in the original parameter distributions caused by the hybrid attack. However, these issues do not hinder the effective identification of the plagiarized model.

To illustrate the effectiveness of the recovery mechanism, we extract watermark data before the attack, after the attack, and after recovery for visual analysis (Fig. 5). The watermark data subjected to the hybrid attack exhibits significant deviations from the original distribution (Fig. 5(a)). We compute key statistics (i.e., minimum, maximum, median, mean, and standard deviation) and visualize them using radar plots (Fig. 5(b)), revealing significant discrepancies between the post-attack and original watermark statistics. The merged watermark data and key exhibit poor compatibility, resulting in degraded performance on the secret dataset. As shown in Table VIII, *AS* fails to verify ownership across all attack scenarios. After recovery, the watermark data closely aligns with the original distribution (Fig. 5(c)). To further illustrate the similarity, we visualize the kernel density estimates of the watermark data, which show substantial overlap and negligible difference (Fig. 5(d)). The recovered watermark data integrates well with the watermark key, achieving strong performance on the secret dataset. As shown in Table VIII, *AS* successfully verified ownership after recovery. Consequently, Fed-RWM demonstrates robustness against diverse attack scenarios and maintains reliable ownership verification.

For the LSTM model employing the *sigmoid* activation function, the performance of the watermarked model exhibits minimal fluctuation after the hybrid attack (Table IX). The ownership verification accuracy slightly decreases from 0.9877 (pre-attack) to 0.9874 (post-attack), yet the ownership remains verifiable.

Additionally, we perform five attack-verification experiments per attack type and define the verification success rate as the proportion of successful verifications post-attack. As shown in Table VIII, all models achieve a verification success

TABLE IX
THE ROBUSTNESS TO HYBRID ATTACKS ON LSTM MODEL

	<i>Acc</i>	<i>AS</i>	Success rate
PF	0.8538	0.9874	1.0
FP	0.8482	0.9874	1.0
Success rate	1.0	-	-

TABLE X
NUMBER OF MODEL PARAMETERS AND PERCENTAGE OF WATERMARKS

	ResNet18	DesNet121	LSTM	CNN
Number of parameters	11,192,019	7,047,754	2,658,945	807,877
Percentage of watermarks	0.0535%	0.0850%	0.2254%	0.7417%

rate of 1.0, corresponding to 100% defense against all attacks. Table IX illustrates that the LSTM model also achieves a verification success rate 100% in all attack scenarios.

5) *Robustness Against Overwrite Attack*: For the overwrite attack, the dishonest client may generate a new watermark and embed it into M using the same method, intending to overwrite the original. We conduct the overwrite attack on four watermarked models under non-IID. The results show that *AS* values of ResNet18, DesNet121, LSTM, and CNN, are 0.9586, 0.9687, 0.9875, and 0.9867, respectively, which do not decrease significantly and still verify the ownership. The underlying cause is that the number of parameters in w_m is much smaller than M (The watermarking percentage in the four models are 0.0850%, 0.0535%, 0.2254% and 0.7417%, respectively, Table X). Thus, the likelihood of the watermark being overwritten is low. Our approach can resist the overwrite attack.

E. Local Model Protection

In Fed-RWM, we embed and train watermark for the global model. During the FL, clients train to obtain local models L , which must also be protected. In this section, we evaluate the watermarks in L . Fig. 3 documents the *AS* during the global model watermark training process (left) and a randomly selected local model verification watermarking process (right). The results indicate that the ownership verification capability improves and stabilizes as training rounds increase. Both global and local models can successfully embed watermarks with similar watermarking performance. Their maximum difference is 0.06%, which is negligible in ownership verification

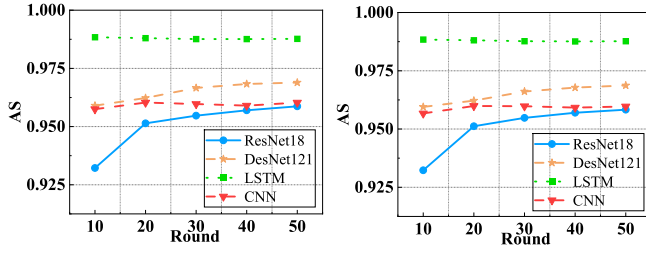


Fig. 6. Left: the global model watermark training process, Right: the local model watermark verification process.

TABLE XI
COMPARISON WITH EXISTING WORKS

Methods	Fine tuning	Pruning	Scaling	Hybrid	SFL	RNN
FedTracker [8]	✓	✓	✗	✗	✗	✗
FedIPR [10]	✓	✓	✗	✗	✓	✓
VPFL [24]	✓	✓	✗	✗	✗	✗
FedMCT [23]	✓	✓	✗	✗	✗	✗
Our	✓	✓	✓	✓	✓	✓

capability. This is because local model training essentially fine-tunes the global model. So, we can easily protect the local model as well.

F. Comparison With Existing Works

We compare Fed-RWM with SOTA white-box watermarking methods [8], [10], [23], [24] for FL models. Among them, [24] is a watermarking scheme applied in IoT. Their embedding processes are all based on enhancements of the work by [11] and are relatively similar. Therefore, our comparison focuses on [10] without loss of generality. According to the experimental evaluation in [10], it is robust to fine-tuning and pruning attacks. Additionally, it also performs well in terms of fidelity and validity. Hence, we compare it in other aspects (Table XI), including robustness against parameter scaling attacks, hybrid attacks, and compatibility with secure federated learning.

Specifically, we train ResNet18 utilizing Fed-RWM and the feature-based approach in [10] to obtain two watermarked models, M and M_c . Before the scaling attack, the accuracy rates of the two models for the main task were 84.74% and 85.60%, with watermark detection rates of 95.87% and 100%, respectively. During the attack, we multiplied some of the convolution and BN layer parameters by the scaling factor and took them negatively. After the attack, the accuracy rates of the two models were 84.85% and 83.51%, respectively, which did not decrease significantly. The watermark detection rate of M was 95.87%, while M_c dropped to 45%. Consequently, M_c can not withstand the scaling attack, which results in severe watermark damage and ownership verification failure. This occurs because the take-negative operation inverts the parameter space w , affecting the bit vector $\hat{B}$, computed from the saved secret matrix x and w . As a result, the stored watermarks B and $\hat{B}$ show significant distance differences, leading to a reduced detection rate. In contrast, Fed-RWM

TABLE XII
ADDITIONAL COMPUTING TIME AND COMMUNICATION OVERHEAD

Model	Embedded training (s)	Verification (s)	Communication (MB)
ResNet18	5.41	53.21	44.85
DesNet121	13.04	360.57	28.70
CNN	3.83	24.83	3.26
LSTM	3.26	1.67	10.65

can still accurately verify the ownership. Since hybrid attacks include parameter scaling, we conclude that M_c cannot defend against them. References [8], [23], [24] embed local watermarks follows the same principle as [10]. Thus, models trained using these methods cannot defend against parameter scaling and hybrid attacks.

Our approach to embed and train watermarks is performed in the initiator, and the server only acts as an aggregation computation, the client only trains the local model. Hence, Fed-RWM avoids dishonest server-client behaviour and adapts to SFL scenarios. References [8], [24] and [23] embed watermarks on the server, which cannot avoid server dishonesty and is not adaptable to SFL. Reference [10] embed watermarks on clients, which cannot avoid dishonest clients embedding malicious watermarks or eliminating watermarks. About the universality of watermarking schemes, our experiments show that Fed-RWM is applicable in both CNN-based and RNN-based models. Furthermore, Fed-RWM can handle tasks involving three types of data: image, traffic, and text. Nevertheless, [8], [24] and [23] do not validate RNN models and only involve the image data-based task. Regarding the watermark embedding position, they all embed the watermark in the BN layer and do not account for models lacking BN layers. However, in our approach, model ownership is protected effectively, irrespective of the presence of BN layers. Notably, the CNN and LSTM models do not contain BN layers in our experiments, while ResNet18 and DesNet121 do.

G. Overhead

1) *Overhead for Watermark Embedding and Verification:* We discuss the additional overhead introduced by our method during watermark embedding training, and verification, as shown in Table XII. The results show that the additional computation time required by our method is negligible during the federated learning process.

During the embedding training phase, the additional overhead primarily arises from training the secret model and embedding the watermark. The secret model, designed as a lightweight architecture with minimal parameters and training data, incurs approximately 3s in training costs. The computational time cost of watermark embedding is predominantly determined by the architectural structure and parameter count of the protected model.

During the verification phase, additional overheads primarily include pruning both the original watermarked model and the suspicious model, computing SVD diagonal matrices for each layer, determining scaling factors, recovering the watermark, and verifying ownership. This is necessary because

the protected model may be vulnerable to scaling attacks and hybrid attacks. Verification time consumption is determined by the model's size and layer count. Although DenseNet121 has fewer parameters than ResNet18, its greater number of layers leads to higher computational times for SVD and scaling factor calculations. Scaling and hybrid attacks are ineffective against models employing sigmoid functions. For such models, additional overhead is primarily associated with ownership verification.

2) *Overhead for Synchronization:* Fed-RWM also affects the synchronization of federated learning. Since our scheme is synchronous federated learning, i.e., in each round of communication, the server side needs to wait until all clients have sent their local models before aggregation. In the traditional scheme, the synchronization time consumption in each round is $T_{max} + 2T_s + T_a$. T_{max} is the maximum duration of local training for all participating clients in this round, T_s is the duration of the model transfer between the server and clients, and T_a is the duration of aggregation. For Fed-RWM, the synchronization time consumption in the watermark embedding round is $T_{max} + 4T_s + T_e + T_a$. T_e is the duration of watermark embedding. In the watermark training round, the initiator only updates the watermark key without updating the watermark, i.e., $\max(T_{max}, T_i + T_w) + 2T_s + T_a$. T_i denotes the duration of the initiator local training for the FL master task, while T_u represents the time required for updating the watermarking task. When the initiator is not selected for the FL master task training in a given round, T_i is 0. It is worth noting that the synchronization time is defined as the time between the start of sending the model to clients in the current round and the start of preparing to send the model in the next round. Compared to the time synchronization overhead of traditional synchronous federated learning, the effect of Fed-RWM on time synchronization is small and tolerable.

3) *Overhead for Federated Communication:* Compared to the baselines, our method introduces additional communication overhead. Specifically, the additional overhead in each round mainly comes from transferring the current global model from the server to the initiator. This process is done to synchronize the watermark and the corresponding key. We quantify these additional overheads by measuring the size of the model files transferred in each round, as shown in Table XII. Although these overheads are incurred, they are tolerable for the current network environment.

3) *Overhead for Federated Communication:* In summary, Fed-RWM is a FL white-box watermarking approach designed to prevent dishonest behaviors on both client and server sides. It surpasses existing SOTA methods in robustness and can be integrated into secure federated learning frameworks. Additionally, its watermark embedding method is generic and independent of model structure, offering broader applicability compared to other approaches. This study implements FL model watermarking in the IoT domain. Our experiments show that Fed-RWM is able to be applied to image, text, and IoT traffic tasks and protect the intellectual property rights of AI models for enterprise or edge device nodes.

VII. CONCLUSION

This paper proposes Fed-RWM, an effective white-box-based watermarking method, to protect the ownership of FL models in IoT. This is the first new method that integrates watermark recovery and simultaneously prevents edge and cloud dishonest behavior. Fed-RWM employs parameter sharing for embedding and verifying the watermark, introduces a new training paradigm to prevent cloud and edge access to watermark information, and incorporates a recovery mechanism to enhance verification robustness. Evaluations of various tasks and settings show that Fed-RWM is effective in terms of ownership verification, fidelity and robustness, etc.

REFERENCES

- [1] V. Demertzi, S. Demertzis, and K. Demertzis, "An overview of privacy dimensions on the Industrial Internet of Things (IIoT)," *Algorithms*, vol. 16, no. 8, p. 378, 2023.
- [2] N. M. Hijazi, M. Aloqaily, M. Guizani, B. Ouni, and F. Karray, "Secure federated learning with fully homomorphic encryption for IoT communications," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 4289–4300, Feb. 2023.
- [3] G. Xu, Z. Zhou, J. Dong, L. Zhang, and X. Song, "A blockchain-based federated learning scheme for data sharing in Industrial Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21467–21478, Dec. 2023.
- [4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," 2016, *arXiv:1602.05629*.
- [5] R. Yang, H. He, Y. Wang, Y. Qu, and W. Zhang, "Dependable federated learning for IoT intrusion detection against poisoning attacks," *Comput. Secur.*, vol. 132, Sep. 2023, Art. no. 103381.
- [6] R. Myrzashova, S. H. Alsamhi, A. V. Shvetsov, A. Hawbani, and X. Wei, "Blockchain meets federated learning in healthcare: A systematic review with challenges and opportunities," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14418–14437, Aug. 2023.
- [7] H. Liu, S. Li, W. Li, and W. Sun, "Efficient decentralized optimization for edge-enabled smart manufacturing: A federated learning-based framework," *Future Gener. Comput. Syst.*, vol. 157, pp. 422–435, Aug. 2024.
- [8] S. Shao, W. Yang, H. Gu, Z. Qin, L. Fan, and Q. Yang, "Fedtracker: Furnishing ownership verification and traceability for federated learning model," *IEEE Trans. Depend. Secure Comput.*, vol. 22, no. 1, pp. 114–131, Jan./Feb. 2025.
- [9] J. Chen, M. Li, Y. Cheng, and H. Zheng, "FedRight: An effective model copyright protection for federated learning," *Comput. Secur.*, vol. 135, Dec. 2023, Art. no. 103504.
- [10] B. Li, L. Fan, H. Gu, J. Li, and Q. Yang, "FedIPR: Ownership verification for federated deep neural network models," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4521–4536, Apr. 2023.
- [11] Y. Uchida, Y. Nagai, S. Sakazawa, and S. Satoh, "Embedding watermarks into deep neural networks," in *Proc. ACM Int. Conf. Multimedia Retrieval*, 2017, pp. 269–277.
- [12] S. Ben Jabra and M. Ben Farah, "Deep learning-based watermarking techniques challenges: A review of current and future trends," *Circuits, Syst., Signal Process.*, vol. 43, pp. 4339–4368, Jul. 2024.
- [13] L. Fan, K. W. Ng, and C. S. Chan, "Rethinking deep neural network ownership verification: Embedding passports to defeat ambiguity attacks," in *Proc. 33rd Conf. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–10.
- [14] H. Chen, B. D. Rouhani, C. Fu, J. Zhao, and F. Koushanfar, "Deepmarks: A secure fingerprinting framework for digital rights management of deep learning models," in *Proc. Int. Conf. Multimedia Retrieval*, 2019, pp. 105–113.
- [15] P. Lv et al., "HufuNet: Embedding the left piece as watermark and keeping the right piece for ownership verification in deep neural networks," 2021, *arXiv:2103.13628*.
- [16] F. Boenisch, "A systematic review on model watermarking for neural networks," *Front. Big Data*, vol. 4, Nov. 2021, Art. no. 729663.
- [17] A. Bansal et al., "Certified neural network watermarks with randomized smoothing," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 1450–1465.

- [18] Y. Lao, P. Yang, W. Zhao, and P. Li, "Identification for deep neural network: Simply adjusting few weights!" in *Proc. IEEE 38th Int. Conf. Data Eng. (ICDE)*, 2022, pp. 1328–1341.
- [19] X. Qi, T. Xie, R. Pan, J. Zhu, Y. Yang, and K. Bu, "Towards practical deployment-stage backdoor attack on deep neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 13347–13357.
- [20] Y. Li, Y. Jiang, Z. Li, and S.-T. Xia, "Backdoor learning: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 1, pp. 5–22, Jan. 2022.
- [21] W. Yang et al., "Watermarking in secure federated learning: A verification framework based on client-side backdoor," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 1, pp. 1–25, 2023.
- [22] B. G. Tekgul, Y. Xia, S. Marchal, and N. Asokan, "WAFFLE: Watermarking in federated learning," in *Proc. 40th Int. Symp. Reliable Distrib. Syst. (SRDS)*, 2021, pp. 310–320.
- [23] Q. Chen, P. Zheng, Y. Du, W. Luo, and H. Liu, "FedMCT: A federated framework for intellectual property protection and malicious client tracking," in *Proc. 16th Int. Conf. Mach. Learn. Comput.*, 2024, pp. 80–86.
- [24] Y. Wu, H. Cheng, L. Guan, P. Liu, F. Chen, and M. Wang, "VPFL: A verifiable property federated learning framework against invisible attacks in distributed IoT," in *Proc. IEEE Int. Conf. High Perform. Comput. Commun., Data Sci. Syst., Smart City Dependabil. Sensor, Cloud Big Data Syst. Appl. (HPCC/DSS/SmartCity/DependSys)*, 2023, pp. 671–678.
- [25] Y. Xiao, H. Shao, J. Lin, Z. Huo, and B. Liu, "BCE-FL: A secure and privacy-preserving federated learning system for device fault diagnosis under non-IID condition in IIoT," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14241–14252, Apr. 2024.
- [26] R. Yang, T. Zhao, F. R. Yu, M. Li, D. Zhang, and X. Zhao, "Blockchain-based federated learning with enhanced privacy and security using homomorphic encryption and reputation," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21674–21688, Jun. 2024.
- [27] A. P. Kalapaaking, I. Khalil, and X. Yi, "Blockchain-based federated learning with SMPC model verification against poisoning attack for healthcare systems," *IEEE Trans. Emerg. Topics Comput.*, vol. 12, no. 1, pp. 269–280, Jan.–Mar. 2023.
- [28] K. W. Church, Z. Chen, and Y. Ma, "Emerging trends: A gentle introduction to fine-tuning," *Natural Lang. Eng.*, vol. 27, no. 6, pp. 763–778, 2021.
- [29] Q. Wang, Z. Pang, W. Liang, J. Zhang, and K. Wang, "Transferable physical layer authentication based on time-varying patterns toward zero training deployment for mobile IIoT devices," *IEEE Trans. Ind. Informat.*, vol. 20, no. 5, pp. 7675–7685, May 2024.
- [30] P. Jain and P. Kar, "Non-convex optimization for machine learning," *Found. Trends Mach. Learn.*, vol. 10, nos. 3–4, pp. 142–363, 2017.
- [31] Y. Xu et al., "Enhancing decentralized federated learning with model pruning and adaptive communication," *IEEE Trans. Ind. Informat.*, vol. 21, no. 1, pp. 70–84, Jan. 2025.
- [32] N. Lukas, E. Jiang, X. Li, and F. Kerschbaum, "SoK: How robust is image classification deep neural network watermarking?" in *Proc. IEEE Symp. Security Privacy (SP)*, 2022, pp. 787–804.
- [33] M. Zolanvari, "WUSTL-IIOT-2021." 2021. [Online]. Available: <https://dx.doi.org/10.21227/yftq-n229>
- [34] A. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proc. 49th Annu. Meeting Assoc. Comput. Linguist., Human Lang. Technol.*, 2011, pp. 142–150.
- [35] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [36] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.
- [37] S. Hochreiter, "Long short-term memory," in *Neural Computation*. Cambridge, MA, USA: MIT-Press, 1997.
- [38] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [39] Y. Adi, C. Baum, M. Cisse, B. Pinkas, and J. Keshet, "Turning your weakness into a strength: Watermarking deep neural networks by backdoor," in *Proc. 27th USENIX Secur. Symp. (USENIX Secur.)*, 2018, pp. 1615–1631.



Xin Tong (Student Member, IEEE) received the M.S. degree in computer science and technology from North China Electric Power University, Beijing, China, in 2023. She is currently pursuing the Ph.D. degree in cyberspace security with the Beijing University of Posts and Telecommunications, Beijing.

Her current research interests include distributed computing, machine learning security, edge computing, and the Internet of Things.



Shike Li (Graduate Student Member, IEEE) received the Ph.D. degree in control science and engineering from North China Electric Power University, Beijing, China, in 2024.

She is currently a Lecturer with the School of Computer and Information Technology, Shanxi University, Taiyuan, China. She has around ten papers in IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, and IEEE INTERNET OF THINGS JOURNAL. Her research interests include Internet of Vehicles, information security, data privacy, and blockchain systems.



Ni Jin received the M.S. degree in mathematics from the Civil Aviation University of China, Tianjin, China, in 2023. She is currently pursuing the Ph.D. degree in cyberspace security with the Beijing University of Posts and Telecommunications, Beijing, China.

Her current research interests include network security, machine learning, and large language models.



Baojiang Cui received the B.S. degree from the Hebei University of Technology, Tianjin, China, in 1994, the M.S. degree from the Harbin Institute of Technology, Harbin, China, in 1998, and the Ph.D. degree in control theory and control engineering from Naikai University, Tianjin, in 2004.

He is currently a Professor with the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, China. He has around 50 papers in IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE

TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, and IEEE TRANSACTIONS ON CONSUMER ELECTRONICS. His main research interests include detection of software, cloud computing, and the Internet of Things.